

ACBDA vs MVC

Two patterns. Two different layers.

10 slides • what each one solves, and when to use it

THE TANGLED METHOD

One function that:

validates the request

runs a SQL query

calls another service over HTTP

applies business rules

formats the JSON response

Change the database schema and this method breaks.

You cannot test it without both a database and a network.

Tangled concerns are the real enemy.

ACBDA

Five roles, one responsibility each.

A Activity - owns the use case, orchestrates the work

C Component - business rules and calculations

B Builder - assembles the final response object

D DAO - all database access for one entity

A Accessor - all calls to external services

Each box has exactly one reason to change.

GET /users/42/orders

Activity -> handle this endpoint end to end

Accessor -> fetch the user from the auth service

DAO -> read orders for user 42 from the DB

Component -> apply discount + tax rules

Builder -> shape the response DTO

Read it as: who does what, in order.

FLOW

Request

- > Activity own the use case
- > Accessor fetch external data
- > DAO read / write
- > Component apply rules
- > Builder assemble response

Response

Wiring is one direction, always.

PROS

- + test each role alone - mock the DAO, no DB needed
- + swap the database by rewriting one class
- + one place holds all SQL for an entity
- + rules live in Components, not controllers
- + teams can work on layers in parallel

Testability is the real prize.

CONS

- five files where one method would do
- indirection: a request touches five classes
- overkill for a simple CRUD endpoint
- needs discipline or the layers blur
- more navigation when debugging

Structure is a cost you pay on purpose.

MVC

Model domain state and rules

View renders it on screen

Controller . translates input, mediates

Wiring: the View depends on the Model.

The Model never depends on the View.

It is a UI pattern - not a whole-app architecture.

Same idea, smaller scope: separate look from data.

THE DIFFERENCE

scope	backend service		presentation tier
roles	5		3
owns	use case		one screen
wiring	request path		view -> model
fixes	tangled backend		tangled UI
breaks	too much ceremony		massive controller

Different problems - not competitors.

PICK THE LAYER THEN THE PATTERN

Building a UI? MVC or MVVM
Building a service? ACBDA, hexagonal, clean
Real products do both - at the same time.

The shared rule: one reason to change per unit,
and dependencies that point one way.

Match the pattern to the layer, not the trend.